



**UNIVERSIDAD
NACIONAL AUTÓNOMA
DE MÉXICO**



**FACULTAD DE INGENIERÍA
INGENIERÍA EN COMPUTACIÓN**

Sistemas Distribuidos

**Mexi Banco: de un monolito a servicios que se encuentran solos
sábado 19 de septiembre (S5)**

NOMBRE DE LA ALUMNA: Muñoz San Agustin Victoria Monserrat

NÚMERO DE CUENTA: 320094187

NOMBRE DEL PROFESOR: Oscar Manuel Ruiz Hurtado

GRUPO: 02

SEMESTRE 2027-1

FECHA DE ENTREGA: sábado 10 de octubre de 2026

PASO 1: La historia (22:45 MX)

```
Victor@bravoservice: ~/v06001 ~/mexi-banco (trabaja-s05)
$ docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
mexi-banco-v06-cuenta-1	mexi-banco-v06-cuenta	"java -jar app.jar"	cuenta	23 seconds ago	Up 9 seconds (health: starting)	0.0.0.0:8081->8080/tcp, [::]:8081->8080/tc
mexi-banco-v06-db-1	postgres:17-alpine	"docker-entrypoint.s..."	db	15 minutes ago	Up 15 minutes (healthy)	5432/tcp
mexi-banco-v06-discovery-1	mexi-banco-v06-discovery	"java -jar app.jar"	discovery	23 seconds ago	Up 20 seconds (healthy)	0.0.0.0:8761->8761/tcp, [::]:8761->8761/tc
mexi-banco-v06-movimiento-1	mexi-banco-v06-movimiento	"java -jar app.jar"	movimiento	23 seconds ago	Up 9 seconds (health: starting)	0.0.0.0:8082->8080/tcp, [::]:8082->8080/tc
mexi-banco-v06-notificacion-1	mexi-banco-v06-notificacion	"java -jar app.jar"	notificacion	23 seconds ago	Up 9 seconds (health: starting)	0.0.0.0:8084->8080/tcp, [::]:8084->8080/tc
mexi-banco-v06-spei-1	mexi-banco-v06-spei	"java -jar app.jar"	spei	23 seconds ago	Up 9 seconds (health: starting)	0.0.0.0:8085->8080/tcp, [::]:8085->8080/tc
mexi-banco-v06-transferencia-1	mexi-banco-v06-transferencia	"java -jar app.jar"	transferencia	23 seconds ago	Up 9 seconds (health: starting)	0.0.0.0:8083->8080/tcp, [::]:8083->8080/tc

¿Qué se rompió al pasar de llamada a método a HTTP?

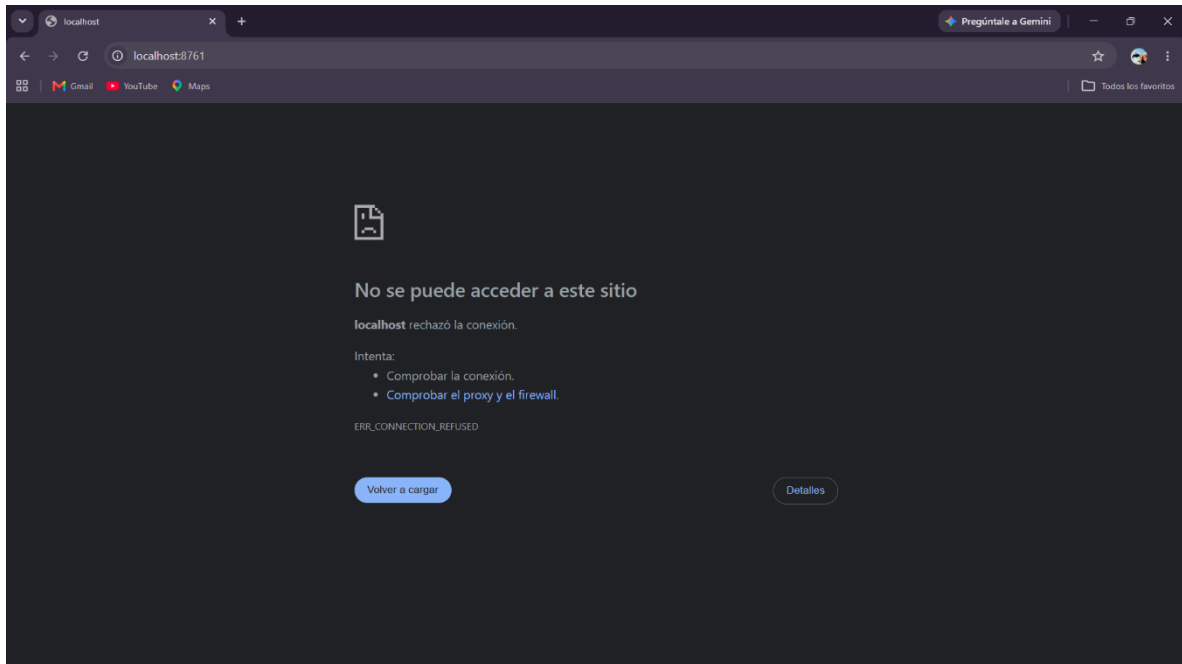
En la arquitectura monolítica (S05), todo el sistema se ejecutaba dentro de un mismo proceso de Java. Una transferencia se realizaba llamando a métodos locales dentro de una única transacción. Si ocurría un fallo al abonar el dinero a la cuenta destino, la base de datos realizaba un rollback de forma automática de toda la operación, garantizando el principio de "todo o nada".

Al separar la aplicación en 5 microservicios independientes con HTTP (S06):

- Pérdida del @Transactional global: Cada microservicio tiene su propia base de datos y sus propias transacciones locales independientes.
- Inconsistencia de datos: Cuando transferencia solicita un cargo a cuenta por HTTP, ese cargo queda confirmado en la base de datos de origen al instante. Si la petición HTTP posterior para realizar el abono en la cuenta destino falla (por red o error del servicio), el cargo inicial ya no se deshace automáticamente.
- Manejo de red y direcciones: Aparecieron problemas de localización y falta de tolerancia a fallos ante caídas de red o de instancias.

PASO 2: Directorio (23:10 MX)

Antes: Al intentar abrir <http://localhost:8761>, el navegador no cargaba la página (ERR_CONNECTION_REFUSED) porque el servidor de descubrimiento aún no estaba activo ni publicado.



Después: Al habilitar @EnableEurekaServer y levantar el contenedor discovery, la página <http://localhost:8761> mostró el panel de Eureka con los 5 servicios registrados en estado UP (1): CUENTA, MOVIMIENTO, NOTIFICACION, SPEI y TRANSFERENCIA.

A screenshot of the Eureka dashboard in a web browser. The dashboard displays various system metrics and a list of registered instances. The 'Instances currently registered with Eureka' section shows a table with columns: Application, AMIs, Availability Zones, and Status. The table lists five applications: CUENTA, MOVIMIENTO, NOTIFICACION, SPEI, and TRANSFERENCIA, all with a status of 'UP (1)'. Below the table, there is a 'General Info' section.

Environment	test	Current time	2026-10-10T05:11:05 +0000
Data center	default	Uptime	00:01
		Lease expiration enabled	true
		Renews threshold	10
		Renews (last min)	10

THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.

DS Replicas

[localhost](#)

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
CUENTA	n/a (1)	(1)	UP (1) - 93481be27616:cuenta:8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - c8650bf9132c:movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - f27626b0f774:notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - 23e271dd5614:spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - e63151df5ee1:transferencia:8080

General Info

Consulta XML del servicio CUENTA

(<http://localhost:8761/eureka/apps/CUENTA>):

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<application>
<name>CUENTA</name>
<instance>
<instanceId>21ec7e7c72e6:cuenta:8080</instanceId>
<hostName>172.18.0.4</hostName>
<app>CUENTA</app>
<ipAddr>172.18.0.4</ipAddr>
<status>UP</status>
<overriddenstatus>UNKNOWN</overriddenstatus>
<port enabled="true">8080</port>
<securePort enabled="false">443</securePort>
<countryId>1</countryId>
<dataCenterInfo class="com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo">
<name>MyOwn</name>
</dataCenterInfo>
<leaseInfo>
<renewalIntervalInSecs>5</renewalIntervalInSecs>
<durationInSecs>15</durationInSecs>
<registrationTimestamp>1791609801992</registrationTimestamp>
<lastRenewalTimestamp>1791611818720</lastRenewalTimestamp>
<evictionTimestamp>0</evictionTimestamp>
<serviceUpTimestamp>1791609801564</serviceUpTimestamp>
</leaseInfo>
<metadata>
<management.port>8080</management.port>
</metadata>
<homePageUrl>http://172.18.0.4:8080</homePageUrl>
<statusPageUrl>http://172.18.0.4:8080/actuator/info</statusPageUrl>
<healthCheckUrl>http://172.18.0.4:8080/actuator/health</healthCheckUrl>
```

```
<vipAddress>cuenta</vipAddress>
<secureVipAddress>cuenta</secureVipAddress>
<isCoordinatingDiscoveryServer>false</isCoordinatingDiscoveryServer>
<lastUpdatedTimestamp>1791609801992</lastUpdatedTimestamp>
<lastDirtyTimestamp>1791609801500</lastDirtyTimestamp>
<actionType>ADDED</actionType>
</instance>
<instance>
<instanceId>92a4974ea0de:cuenta:8080</instanceId>
<hostName>172.18.0.11</hostName>
<app>CUENTA</app>
<ipAddr>172.18.0.11</ipAddr>
<status>UP</status>
<overriddenstatus>UNKNOWN</overriddenstatus>
<port enabled="true">8080</port>
<securePort enabled="false">443</securePort>
<countryId>1</countryId>
<dataCenterInfo class="com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo">
<name>MyOwn</name>
</dataCenterInfo>
<leaseInfo>
<renewalIntervalInSecs>5</renewalIntervalInSecs>
<durationInSecs>15</durationInSecs>
<registrationTimestamp>1791609646585</registrationTimestamp>
<lastRenewalTimestamp>1791611817267</lastRenewalTimestamp>
<evictionTimestamp>0</evictionTimestamp>
<serviceUpTimestamp>1791609646071</serviceUpTimestamp>
</leaseInfo>
<metadata>
<management.port>8080</management.port>
</metadata>
```

```
<homePageUrl>http://172.18.0.11:8080/</homePageUrl>
<statusPageUrl>http://172.18.0.11:8080/actuator/info</statusPageUrl>
<healthCheckUrl>http://172.18.0.11:8080/actuator/health</healthCheckUrl>
<vipAddress>cuenta</vipAddress>
<secureVipAddress>cuenta</secureVipAddress>
<isCoordinatingDiscoveryServer>false</isCoordinatingDiscoveryServer>
<lastUpdatedTimestamp>1791609646585</lastUpdatedTimestamp>
<lastDirtyTimestamp>1791609645262</lastDirtyTimestamp>
<actionType>ADDED</actionType>
</instance>
<instance>
<instanceId>96eb788c30ff:cuenta:8080</instanceId>
<hostName>172.18.0.10</hostName>
<app>CUENTA</app>
<ipAddr>172.18.0.10</ipAddr>
<status>UP</status>
<overriddenstatus>UNKNOWN</overriddenstatus>
<port enabled="true">8080</port>
<securePort enabled="false">443</securePort>
<countryId>1</countryId>
<dataCenterInfo class="com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo">
<name>MyOwn</name>
</dataCenterInfo>
<leaseInfo>
<renewalIntervalInSecs>5</renewalIntervalInSecs>
<durationInSecs>15</durationInSecs>
<registrationTimestamp>1791609649142</registrationTimestamp>
<lastRenewalTimestamp>1791611815655</lastRenewalTimestamp>
<evictionTimestamp>0</evictionTimestamp>
<serviceUpTimestamp>1791609648628</serviceUpTimestamp>
</leaseInfo>
```

```
<metadata>
<management.port>8080</management.port>
</metadata>
<homePageUrl>http://172.18.0.10:8080/</homePageUrl>
<statusPageUrl>http://172.18.0.10:8080/actuator/info</statusPageUrl>
<healthCheckUrl>http://172.18.0.10:8080/actuator/health</healthCheckUrl>
<vipAddress>cuenta</vipAddress>
<secureVipAddress>cuenta</secureVipAddress>
<isCoordinatingDiscoveryServer>false</isCoordinatingDiscoveryServer>
<lastUpdatedTimestamp>1791609649142</lastUpdatedTimestamp>
<lastDirtyTimestamp>1791609648298</lastDirtyTimestamp>
<actionType>ADDED</actionType>
</instance>
</application>
```

En el compose, transferencia sigue llamando a CUENTA_URL:

<http://cuenta:8080>. ¿Alguien está usando el directorio para encontrar a cuenta?

En este punto, nadie está usando el directorio aún para comunicarse con cuenta. La resolución de <http://cuenta:8080> la realiza directamente el servidor DNS interno de Docker Compose, no Eureka. Eureka solo está registrando la información, pero los servicios todavía no consultan al directorio.

PASO 3: Gateway (23:16 MX)

Resultados de las pruebas con curl:

Llamada directa interna a movimiento (curl -i -X POST

<http://localhost:8082/movimientos>):

```
victo@brannvicc MINGW64 ~/mexi-banco (trabajo-s07)
$ curl -i -X POST http://localhost:8082/movimientos
HTTP/1.1 400
Content-Type: application/problem+json
Transfer-Encoding: chunked
Date: Sat, 10 Oct 2026 05:15:07 GMT
Connection: close

{"type":"about:blank","title":"Bad Request","status":400,"detail":"Failed to read request","instance":"/movimientos"}
```

Explicación: La llamada llega directamente al microservicio de movimiento sin pasar por ningún filtro. Nos responde 400 porque faltan los datos del cuerpo en el POST, demostrando que el endpoint interno está desprotegido en su puerto directo.

Acceso por el Gateway (<http://localhost:8080>):

- Prueba A (`curl -i http://localhost:8080/api/cuentas/000`):

```
victo@brannvicc MINGW64 ~/mexi-banco (trabajo-s07)
$ curl -i http://localhost:8080/api/cuentas/000
HTTP/1.1 404 Not Found
transfer-encoding: chunked
X-Instancia: 93481be27616
Content-Type: application/problem+json
Date: Sat, 10 Oct 2026 05:16:04 GMT

{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 000","instance":"/cuentas/000"}
```

El microservicio cuenta. El Gateway enrutó correctamente la petición hasta cuenta, y fue el microservicio de negocio el que contestó "No existe la CLABE 000".

- Prueba B (`curl -i -X POST http://localhost:8080/api/movimientos`):

```
victo@brannvicc MINGW64 ~/mexi-banco (trabajo-s07)
$ curl -i -X POST http://localhost:8080/api/movimientos
HTTP/1.1 404 Not Found
Content-Type: application/json
Content-Length: 129
```

El API Gateway. Bloqueó la petición en la entrada porque en el archivo `application.yml` configuramos la ruta de `/api/movimientos` para aceptar únicamente el método GET.

Alta y consulta de cuenta a través del Gateway:

- Alta (201 Created): Se ejecutó `curl -X POST http://localhost:8080/api/cuentas` con la CLABE única S071791609385, devolviendo un estado 201 Created y JSON con saldo inicial de 0.

```
victo@brannvicc MINGW64 ~/mexi-banco (trabajo-s07)
$ LAB_CLABE="S07$(date +%s)"
curl -i -X POST http://localhost:8080/api/cuentas \
-H 'Content-Type: application/json' \
-d '{"clabe":"$LAB_CLABE","titular":"S07","saldoInicial":0}'
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: 93481be27616
Content-Type: application/json
Date: Sat, 10 Oct 2026 05:16:25 GMT
```

- Consulta (200 OK): Se ejecutó curl
http://localhost:8080/api/cuentas/S071791609385, devolviendo 200 OK y devolviendo la información en formato JSON a través del puerto único 8080.

```
victo@brannvicc MINGW64 ~/mexi-banco (trabajo-s07)
$ URL="http://localhost:8080/api/cuentas/$LAB_CLABE"
curl -i "$URL"
echo "$URL"
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 93481be27616
Content-Type: application/json
Date: Sat, 10 Oct 2026 05:16:34 GMT
{"id":1,"clabe":"S071791609385","titular":"S07","saldo":0.00}http://localhost:8080/api/cuentas/S071791609385
```

Paso 4 y 5: Copias y caída (23:20 MX)

Al configurar deploy: replicas: 3 en cuenta dentro del docker-compose.yml y remover la publicación directa de sus puertos de la laptop:

```
victo@brannvicc MINGW64 ~/mexi-banco (trabajo-s07)
$ docker ps --filter name=cuenta --format '{{.ID}} {{.Names}} {{.Ports}}'
21ec7e7c72e6 mexi-banco-v06-cuenta-1 8080/tcp
96eb788c30ff mexi-banco-v06-cuenta-2 8080/tcp
92a4974ea0de mexi-banco-v06-cuenta-3 8080/tcp
```

En http://localhost:8761, el servicio CUENTA cambió a UP (3).

Instances currently registered with Eureka			
Application	AMIs	Availability Zones	Status
CUENTA	n/a (3)	(3)	UP (3) - 21ec7e7c72e6:cuenta:8080 , 92a4974ea0de:cuenta:8080 , 96eb788c30ff:cuenta:8080
GATEWAY	n/a (1)	(1)	UP (1) - dbc9e1ca7bba:gateway:8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - 70d829829aab:movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - 7536f2214355:notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - e3fa83bb32ab:spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - 826cbf6826f7:transferencia:8080

Al ejecutar las 9 consultas consecutivas en la terminal:

```
victo@brannvicc MINGW64 ~/mexi-banco (trabajo-s07)
$ for i in {1..9}; do curl -sS -D - -o /dev/null "$URL" | grep -i '^X-Instancia'; done
X-Instancia: 21ec7e7c72e6
X-Instancia: 92a4974ea0de
X-Instancia: 96eb788c30ff
X-Instancia: 21ec7e7c72e6
X-Instancia: 92a4974ea0de
X-Instancia: 96eb788c30ff
X-Instancia: 21ec7e7c72e6
X-Instancia: 92a4974ea0de
X-Instancia: 96eb788c30ff
```

Se observó que los valores de X-Instancia alternaron ordenadamente entre los IDs de las tres réplicas (cuenta-1, cuenta-2 y cuenta-3), confirmando el algoritmo Round Robin de Spring Cloud Gateway.

Simulación de caída con docker kill:

Predicción previa: Las consultas enviadas a la réplica muerta fallarán con código 500/503 temporalmente, mientras que las enviadas a las copias vivas responderán 200. Los errores desaparecerán cuando Eureka remueva la instancia caída.

```
MINGW64/c/Users/victo/mexi-banco
$ for i in {1..60}; do
  echo "$(date +%T) $(curl -s -o /dev/null -w '%{http_code}' --max-time 3 "$URL")"
  sleep 1
done
23:21:59 200
23:22:00 200
23:22:01 200
23:22:02 200
23:22:04 200
23:22:05 200
23:22:06 200
23:22:07 200
23:22:08 500
23:22:11 200
23:22:13 500
23:22:16 200
23:22:17 500
23:22:20 200
23:22:21 200
23:22:23 200
23:22:24 200
23:22:25 500
23:22:28 200
23:22:29 500
23:22:33 200
23:22:34 200
23:22:35 500

MINGW64/c/Users/victo
$ date +%T # anota la hora exacta del apagado
docker kill mexi-banco-v06-cuenta-1
23:22:07
mexi-banco-v06-cuenta-1

victo@brannvict MINGW64 ~
$ date +%T # anota la hora exacta del apagado
23:22:39

victo@brannvict MINGW64 ~
$ docker kill mexi-banco-v06-cuenta-1
Error response from daemon: cannot kill container: mexi-banco-v06-cuenta-1: container 21ec7e7c72e6584679c4c4fe2c73e810283ce8170c815111be556aff60e23c is not running

victo@brannvict MINGW64 ~
$ date +%T # anota la hora exacta del apagado
23:22:46
docker kill mexi-banco-v06-cuenta-1
Error response from daemon: cannot kill container: mexi-banco-v06-cuenta-1: container 21ec7e7c72e6584679c4c4fe2c73e810283ce8170c815111be556aff60e23c is not running

victo@brannvict MINGW64 ~
$ docker start mexi-banco-v06-cuenta-1
mexi-banco-v06-cuenta-1

victo@brannvict MINGW64 ~
$ |
```

Application	AMIs	Availability Zones	Status
CUENTA	n/a (2)	(2)	UP (2) - 92a4974ea0de:cuenta:8080 , 96eb788c30ff:cuenta:8080
GATEWAY	n/a (1)	(1)	UP (1) - dbc9e1ca7bba:gateway:8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - 70d829829aab:movimiento:8080

- Hora del apagado de la instancia (docker kill mexi-banco-v06-cuenta-1): 23:22:07
- Hora de salida del directorio (paso a UP (2) en tablero): 23:22:39 (~32 segundos después).
- Cantidad de errores observados: Se observaron 7 errores 500 intercalados en las consultas de un total de 45 peticiones.

¿Por qué ocurre esto? Porque el Gateway consulta la lista de instancias a Eureka de forma periódica mediante una caché interna. Durante la ventana de tiempo (~30 s) previa a la expiración del plazo de expiración y refresco de caché, el Gateway continúa enviando 1 de cada 3 peticiones a la réplica muerta.

Se recupera con:

```
victo@brannvicc MINGW64 ~/mexi-banco (trabajo-s07)
$ docker start mexi-banco-v06-cuenta-1
mexi-banco-v06-cuenta-1
```

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
CUENTA	n/a (3)	(3)	UP (3) - 21ec7e7c72e6:cuenta:8080 , 92a4974ea0de:cuenta:8080 , 96eb788c30ff:cuenta:8080
GATEWAY	n/a (1)	(1)	UP (1) - dbc9e1ca7bba:gateway:8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - 70d829829aab:movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - 7536f2214355:notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - e3fa83bb32ab:spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - 826cbf6826f7:transferencia:8080

Cierre

1. El directorio resuelve la localización dinámica de servicios en la red, permitiendo que las aplicaciones se encuentren por nombre sin tener que escribir direcciones IP o puertos a mano.
2. El gateway resuelve disponer de una puerta de entrada única e unificada hacia el exterior, aplicando filtros, ocultando puertos internos y gestionando reglas de acceso HTTP.
3. El balanceador resuelve la distribución equilibrada del tráfico de peticiones entre múltiples réplicas disponibles y la tolerancia a fallos ante caídas.
4. Cuando murió una copia vi errores de HTTP 500/503 intercalados durante aproximadamente 30 segundos porque el servidor Eureka y la caché del Gateway tardan un periodo de tiempo determinado en detectar la falta de latidos (heartbeats) y remover la instancia inactiva del registro.

Tarea

```
MINGW64 ~/Users/victo/mexi-banco
victo@victorio:~/Users/victo/mexi-banco (trabajo-s07)
$ ./demo-v06.sh

--- 0. Esperar a que el directorio conozca a todos y la puerta encuentre a cuenta ---
$ lista en 2 s. lo que sabe el directorio (http://localhost:8080/sureka/apps):
Cuenta      3 instancia(s)
GATEWAY     1 instancia(s)
MOVIMIENTO  1 instancia(s)
NOTIFICACION 1 instancia(s)
SPEI        1 instancia(s)
TRANSFERENCIA 1 instancia(s)
+ 404 de cuenta (no 303 del gateway) para una CLAVE que no existe: la puerta ya sabe llegar

--- 1. Abrir la cuenta de Ana con 1000 (por la puerta: POST /api/cuentas) ---
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clave":"002180009161038801","titular":"Ana","saldoInicial":1000}'
[HTTP 201]
+ el gateway quito /api y lo mando a /h//cuenta: a una de sus tres replicas

--- 2. Abrir la cuenta de Beto con 500 (misma puerta) ---
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clave":"002180009161038802","titular":"Beto","saldoInicial":500}'
[HTTP 201]

--- 3. Transferir 200 de Ana a Beto (misma puerta: POST /api/transferencias) ---
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009161038801","claveDestino":"002180009161038802","monto":200}'
[HTTP 201]
+ transferencia llamo a http://cuenta (con nombre, sin puerta). A que replicas fue cada llamada:
cuenta-3 GET /cuentas/002180009161038801 atendida por 92a4974ea0de
cuenta-2 GET /cuentas/002180009161038802 atendida por 96eb788c30ff
cuenta-1 POST /cuentas/002180009161038801/cargos atendida por 21ec7e7c72e6
cuenta-3 POST /cuentas/002180009161038802/abonos atendida por 92a4974ea0de
+ consulta de origen y destino, cargo y abono: cada llamada le reparte el balanceador de transferencia

--- 4. Consultar esa transferencia ---
$ curl http://localhost:8080/api/transferencias/2
{"id":2,"claveOrigen":"002180009161038801","claveDestino":"002180009161038802","monto":200.00,"fecha":"2026-10-10T05:33:14.149374Z"}
[HTTP 200]

--- 5. Saldos: el dinero no se crea ni se pierde ---
$ curl http://localhost:8080/api/cuentas/002180009161038801
{"id":6,"clave":"002180009161038801","titular":"Ana","saldo":800.00}
[HTTP 200]
$ curl http://localhost:8080/api/cuentas/002180009161038802
{"id":7,"clave":"002180009161038802","titular":"Beto","saldo":700.00}
[HTTP 200]
+ Ana 800.00 + Beto 700.00 = 1500.00 (antes: 1000 + 500 = 1500). Esperado: 800 y 700

--- 6. Estado de cuenta de Ana y avisos de Beto (GET /api/movimientos, GET /api/notificaciones) ---
$ curl http://localhost:8080/api/movimientos?clave=002180009161038801
{"id":11,"claveCuenta":"002180009161038801","tipo":"TRANSFERENCIA_ENVIADA","monto":200.00,"saldoResultante":800.00,"fecha":"2026-10-10T05:33:14.118512Z","referencia":"a 002180009161038802},{\"id\":\"claveCuenta\":\"002180009161038801\",\"tipo\":\"DEPOSITO\",\"monto\":1000.00,\"saldoResultante\":1000.00,\"fecha\":\"2026-10-10T05:33:12.274318Z\",\"referencia\":\"saldo inicial\"}]
[HTTP 200]
$ curl http://localhost:8080/api/notificaciones?clave=002180009161038802
{"id":6,"claveCuenta":"002180009161038802","mensaje":"TRANSFERENCIA RECIBIDA por 200, saldo resultante 700.00","fecha":"2026-10-10T05:33:14.330144Z"}
[HTTP 200]

--- 7. SPEI de 50 con Idempotency-Key, dos veces (POST /api/spei) ---
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91610388' -d '{"claveOrigen":"002180009161038801","bancoDestino":"BANCO-X","claveDestino":"01218000000000000009","monto":50}'
[HTTP 201]

--- 8. Round robin: 9 consultas seguidas a la misma cuenta, por la misma puerta ---
consulta 1 X-Instancia: 21ec7e7c72e6 (mexi-banco-v06-cuenta-1)
consulta 2 X-Instancia: 92a4974ea0de (mexi-banco-v06-cuenta-3)
consulta 3 X-Instancia: 96eb788c30ff (mexi-banco-v06-cuenta-2)
consulta 4 X-Instancia: 21ec7e7c72e6 (mexi-banco-v06-cuenta-1)
consulta 5 X-Instancia: 92a4974ea0de (mexi-banco-v06-cuenta-3)
consulta 6 X-Instancia: 96eb788c30ff (mexi-banco-v06-cuenta-2)
consulta 7 X-Instancia: 92a4974ea0de (mexi-banco-v06-cuenta-3)
consulta 8 X-Instancia: 21ec7e7c72e6 (mexi-banco-v06-cuenta-1)
consulta 9 X-Instancia: 96eb788c30ff (mexi-banco-v06-cuenta-2)
+ 3 replicas distintas en 9 consultas: 0 veces se repitió la misma dos veces seguidas
+ el cliente no sabe cuantas replicas hay ni donde estan: solo conoce el 8080

--- 9. Los endpoints internos no tienen puerta ---
$ curl -X POST http://localhost:8080/api/cuentas/002180009161038801/abonos -H 'Content-Type: application/json' -d '{"monto":1000000}'
{"timestamp":"2026-10-10T05:33:23.647400Z","path":"/api/cuentas/002180009161038801/abonos","status":404,"error":"Not Found","requestId":"058e2de2-3777"}
[HTTP 404]
+ 404: el gateway no tiene ruta para /cargos ni /abonos. Adentro de la red siguen abiertos (seguridad: mas adelante)

--- 10. Errores de negocio, por la puerta, con el mismo código que dio cuenta ---
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009161038801","claveDestino":"002180009161038899","monto":10}'
{"type":"about:blank","title":"Not Found","status":404,"detail":"no existe la CLAVE 002180009161038899","instance":"transferencias","servicio":"cuenta"}
[HTTP 404]
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009161038801","claveDestino":"002180009161038802","monto":999999}'
{"type":"about:blank","title":"Unprocessable Entity","status":422,"detail":"Saldo insuficiente en la CLAVE 002180009161038801","instance":"transferencias","servicio":"cuenta"}
[HTTP 422]

--- 11. Los saldos no se movieron con los errores ---
+ Ana 750.00 (esperado 750) + Beto 700.00 (esperado 700) = 1450.00; SPEI se llevo 50 a otro banco

Fin. Para ver cuanto tarda el directorio en enterarse de una caída: ./demo-v06.sh --latido

victo@victorio:~/Users/victo/mexi-banco (trabajo-s07)
$ docker compose logs cuenta | grep -E 'cargos|abonos'
cuenta-2 | 2026-10-10T05:30:48.168Z INFO 1 --- [cuenta] [nio-8080-exec-8] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009161024001/cargos atendida por 96eb788c30ff
cuenta-3 | 2026-10-10T05:27:34.219Z INFO 1 --- [cuenta] [nio-8080-exec-7] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009161004501/cargos atendida por 92a4974ea0de
cuenta-1 | 2026-10-10T05:27:29.956Z INFO 1 --- [cuenta] [nio-8080-exec-3] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009161004501/cargos atendida por 21ec7e7c72e6
cuenta-1 | 2026-10-10T05:27:30.005Z INFO 1 --- [cuenta] [nio-8080-exec-4] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009161004502/abonos atendida por 21ec7e7c72e6
cuenta-1 | 2026-10-10T05:27:40.641Z INFO 1 --- [cuenta] [nio-8080-exec-7] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009161004501/cargos atendida por 21ec7e7c72e6
cuenta-3 | 2026-10-10T05:33:14.066Z INFO 1 --- [cuenta] [nio-8080-exec-5] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009161038802/abonos atendida por 92a4974ea0de
cuenta-3 | 2026-10-10T05:33:18.030Z INFO 1 --- [cuenta] [nio-8080-exec-4] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009161038801/cargos atendida por 92a4974ea0de
cuenta-1 | 2026-10-10T05:33:14.047Z INFO 1 --- [cuenta] [nio-8080-exec-4] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009161038801/cargos atendida por 21ec7e7c72e6
cuenta-1 | 2026-10-10T05:33:23.993Z INFO 1 --- [cuenta] [nio-8080-exec-5] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009161038801/cargos atendida por 21ec7e7c72e6
```

Verificación de saldos: Ana finalizó con \$750.00 y Beto con \$700.00.

Conclusión: La anotación `@LoadBalanced` en `CientesHttp.java` del servicio `transferencia` logró que las peticiones internas de cargos y abonos para una misma operación fueran atendidas por réplicas distintas en orden Round Robin.